

Proyecto - Fase #3 - Tabla de símbolos y comprobación de tipos

MiniLang

Fecha de entrega: lunes 19 de mayo, 19:30 horas

1. Objetivo del Proyecto

Entender las fases de proceso de compilación a través de la creación de un compilador, el estudiante deberá crear de forma incremental el compilador y aplicar todos los conocimientos vistos en clase para la solución del problema.

2. Contexto

En esta fase implementará el análisis semántico para MiniLang, integrándolo con el lexer y parser de las fases anteriores. El compilador debe validar los tipos de datos y comprobar que las operaciones sean válidas semánticamente. Además, en esta fase ya se implementa la estructura de tabla de símbolos para llevar un control de todos los tokens que se van analizando. Se deben reportar errores semánticos con recuperación para seguir analizando el resto del archivo.

3. Objetivos de la fase

La tercera fase del proyecto consistirá en generar la tabla de símbolos, realizar la asignación de valores a variables y constantes, comprobar los tipos de ciertas expresiones para los lenguajes que están trabajando.

4. Alcance e Implementación

Tabla de símbolos

La tercera fase debe generar la tabla de símbolos para el lenguaje minilang. Como la tabla se va generando y eliminando en forma dinámica, se deberá ir manejando y generando un archivo de salida que contenga la tabla completa. La estructura de la tabla se deja a discreción del alumno, pero se calificará la presentación de la misma, así como la mejor manera de almacenar y representar los elementos en la tabla.

Asignación de variables y constantes

Durante esta fase también debe realizarse las operaciones y la asignación de valores de las constantes y variables en el lenguaje. De tal manera que, si tenemos una definición de constante de la siguiente forma:

```
const int a;  
a = 35 * 2;
```

El compilador debe almacenar la constante en la tabla de símbolos y el valor de la misma debe indicar 70. También se podrán hacer operaciones y referencias a valores de constantes y variables ya definidas, es decir:

```
double circun;  
const double pi;  
const double radio;  
pi = 3.1416; /*Se guarda la constante pi con valor 3.1416 */  
radio = 20.5; /* Se guarda la constante radio con valor 20.5 */  
circun = pi*2*radio; /*Se guarda la variable con valor 128.8056*/
```

Comprobación de tipos

El compilador debe evaluar los tipos de las variables, constantes y parámetros de los procedimientos y funciones. Deberá indicar si las operaciones de expresiones, asignación y paso de parámetros está permitida en el lenguaje según los tipos de dato que se manejen.

Por ejemplo, se lee el siguiente archivo de prueba:

```
class Program {  
  
    int f1(int a, int b, int c){  
        if (a>b) c=2; else c=3;  
    }  
  
    void main(){  
        int m1,  
        int m2;  
        int m3;  
        Parser MyParser;  
        MyParser.f1(1,2,m3); /* Paso de parámetros correcto */  
        MyParser.f1(1,"2.2",m3); /*Error de argumento 2 inválido*/  
    }  
}
```

El resultado en pantalla debería ser, por ejemplo:

```
line 11, column 26: **ERROR** An argument for function f1 is invalid
```

Sin embargo, su compilador debe considerar la coerción de tipos: Por ejemplo:

```
int A;  
double B;  
A = 9;  
B = 34.35 * A;
```

Lo anterior es un ejemplo de coerción de tipos y es válido ya que debería poder convertir el valor entero a un doble. Pero si tenemos:

```
string A;  
int B;  
A = "hola";  
B = 34 * A;
```

En este caso, el compilador debe indicar que no es válida la operación por tipos no compatibles:
line 2, column 9: ****ERROR**** Cannot operate type 'int' and 'string'

De encontrar errores, el compilador debe continuar hasta el final de archivo. Los errores deberá reportarlos en pantalla indicando:

Línea y columna, Error: <descripción del error>.

De no hallar errores en el archivo de entrada, indicarlo también en pantalla. Su compilador siempre deberá poder reconocer errores léxicos o sintácticos en los archivos de entrada.

5. Entradas, salidas, set de pruebas y entregables

- El programa debe codificarse en lenguaje de programación a su elección: Java, C#, Python. (Se recomienda C#). El ejecutable o script debe ser llamado **minilang** y debe pedir el archivo de entrada.
- Entrada: archivo con extensión **.mlng** con código MiniLang.
- Si el archivo es semánticamente correcto: imprimir OK en pantalla. Si hay errores: imprimir la lista de errores y continuar hasta EOF.
- Entregables:
 - Programa fuente del compilador, que devuelva mensaje de éxito o listado de errores, debidamente documentado internamente
 - Set de casos de prueba utilizados con sus respectivos archivos y resultados esperados.
- Set de pruebas: Se deben entregar al menos 8 pruebas, en las cuales se debe demostrar cómo funcionará su lenguaje, las pruebas deben tener:
 - Prueba 1: Un programa que imprima un "Hola Mundo".
 - Prueba 2: Un programa que realice una operación aritmética básica.
 - Prueba 3: Un programa que demuestre el uso del input de datos y de acuerdo con la entrada del usuario decidir el flujo del programa a través de un if
 - Prueba 4: Definición de una función que reciba parámetros y devuelva un resultado, la función debe ser llamada desde el programa principal.
 - Prueba 5: Programa que contenga definición de variables, entrada y salida de datos, y que utilice el bucle "while"
 - Prueba 6: Prueba negativa con un fallo léxico.

- o Prueba 7: Prueba negativa con un fallo de sintáxis.
- o Prueba 8: Prueba negativa que mostrará cuando ocurra un fallo semántico (Por ejemplo, una variable no declarada)

6. Sobre la entrega

- El proyecto se entregará en un repositorio en GitHub. El código fuente a evaluar será el que esté subido en la sección de entrega del portal. **No se aceptarán cambios en el momento de la calificación.**
- En el repositorio deberá incluir un archivo README donde explique todo el funcionamiento, decisiones de diseño, estrategias de comprobación de tipos usadas, estructura de la tabla de símbolos, cómo le da mantenimiento y su estrategia de errores semánticos.
- Aspectos por evaluar
 - o Adecuada aplicación de los conocimientos vistos en clase
 - o Calidad de la documentación: ortografía, orden, limpieza y que esté completa.
 - o Calidad de la solución propuesta: que solucione el problema (que haga lo que requiere el enunciado) en forma eficaz.
 - o Funcionalidad del programa: debe cumplir a cabalidad con todos los requerimientos.
 - o Evidencia de la creación del programa y dominio de los conceptos utilizados.
 - o Crear repositorio en github y agregar el usuario: diana1190.
 - o Creatividad.
 - o Commits en github de TODOS los integrantes del equipo, deben ser commits de calidad, no comentarios o clases sin desarrollar, NO HAY EXCUSA DE QUE GIT NO FUNCIONA EN UNA COMPUTADORA.

7. Rúbrica de evaluación (100 puntos)

Criterio	Resultado	Descripción	Ponderación
Solución entregada en el portal	Sí / No	Si no está en el portal no se califican los demás criterios y se obtiene 0 puntos	NA
Solución compila	Sí / No	Si la solución no compila no se califican los demás criterios y se obtiene 0 puntos	NA
Estructura de la tabla de símbolos	Sí / No	Estructura definida, documentada y funcional. Es eficiente y se guarda en un archivo de salida aparte.	25
Cobertura de casos de prueba utilizados	Sí / No	Genera los resultados esperados de cada archivo de prueba	35

		(Se podrá modificar los archivos de prueba durante la calificación o pedir que se ejecuten adicionales)	
Defensa de solución	Sí / No	Explica de forma clara y razonada el análisis y lógica de la solución. Evidencia conocimiento y dominio sobre el código presentado.	20
Calidad de documentación	Sí / No	README actualizado, instrucciones de ejecución y decisiones técnicas. El código está documentado.	10
Versionamiento de código	Sí / No	Utiliza Github para las versiones del proyecto	10

¡Buena suerte!